AI-Generated UGC Human Realism Enhancement System

Final Project Report

1. Problem Statement

AI-generated UGC (User-Generated Content) videos used for marketing and ads often look artificial — stiff body movement, unnatural eye gaze, fixed expressions, and inconsistent motion. This makes AI-generated ad content easy to spot as fake, reducing viewer trust. The company wants a system that automatically checks generated videos and tells them exactly what to fix.

2. Objective

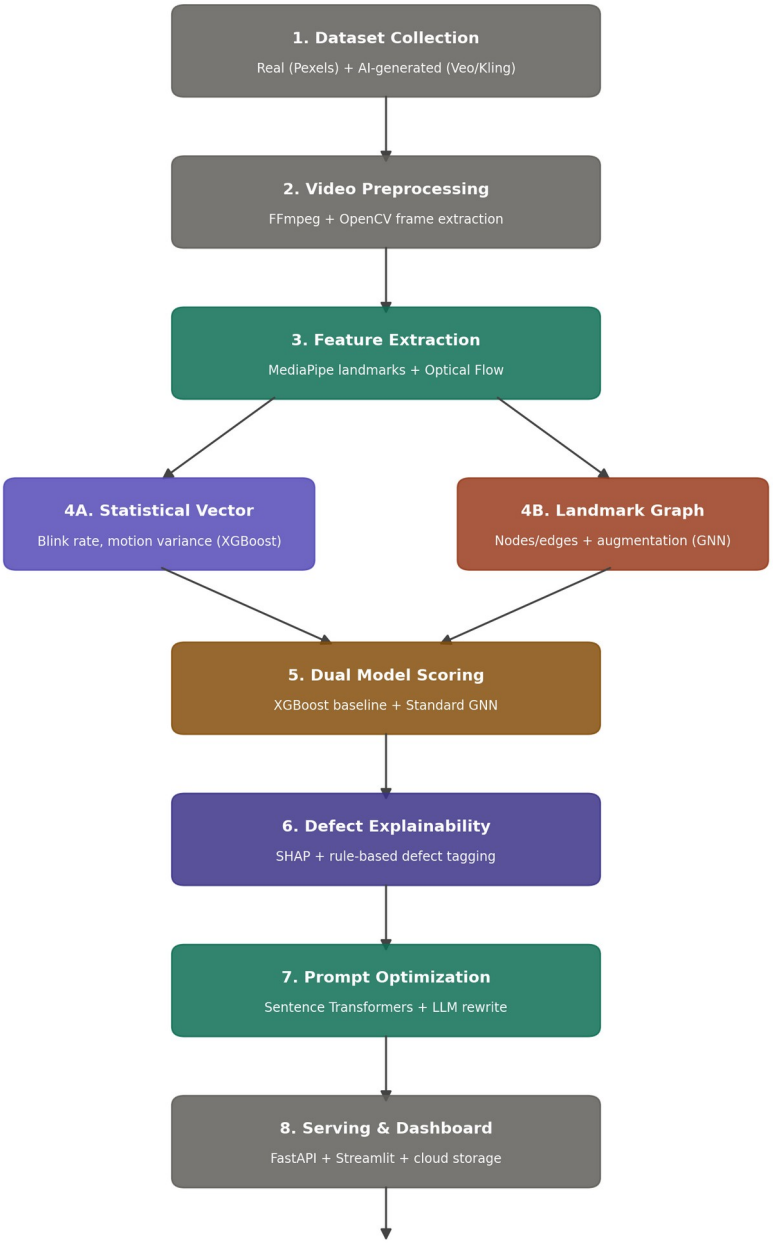
Build an ML-based evaluator that scores how human-like an AI-generated UGC video looks, explains which specific defects are pulling the score down, and automatically rewrites the generation prompt to fix those defects — reducing manual trial-and-error in producing client-ready AI video ads.

3. Proposed Solution

The system acts as a quality-control layer sitting between the video generator and the client. It does not generate videos itself — it evaluates them, explains defects, and improves the prompt so the next generation attempt looks more realistic.

- **Input:** user prompt + AI-generated UGC video
- **Output:** realism score + defect explanation + improved prompt

4. System Flowchart



Loop: improved prompt -> regenerate video -> re-score -> repeat until target reached

5. Step-by-Step Explanation

- **1. Dataset collection:** gather real human UGC videos (Pexels/Pixabay) and AI-generated videos (Veo/Kling/Nano Banana), balanced across regions and settings.
- **2. Video preprocessing:** FFmpeg and OpenCV split each video into frames and normalize FPS/resolution for consistent analysis.
- **3. Feature extraction:** MediaPipe extracts 468 face and 33 pose landmarks per frame; optical flow measures pixel motion between frames.
- **4A. Statistical vector path:** landmark and motion data is summarized into numeric features (blink rate, gesture symmetry) for XGBoost.
- **4B. Landmark graph path:** landmarks are structured as a graph (points as nodes, skeletal links as edges) and augmented for the GNN.
- **5. Dual model scoring:** XGBoost (baseline) and a Standard GNN (primary model) each predict a realism score; results are compared side by side.

- **6. Defect explainability:** SHAP identifies which features caused a low score, and a rule-based layer converts this into plain-language defects.
- **7. Prompt optimization:** detected defects are passed to an LLM (Llama/Mistral), which rewrites the original prompt to correct them.
- **8. Serving and dashboard:** the full pipeline is wrapped in a FastAPI backend with a Streamlit dashboard showing scores and suggestions.
- **Loop:** the improved prompt is used to regenerate the video, which is re-scored — this repeats until the target realism score is reached.

6. Why This Model Combination (XGBoost + GNN)

- **XGBoost:** fast, reliable, works well on tabular feature vectors, needs no GPU — used as a safety-net baseline.
- **Standard GNN:** landmarks naturally form a skeletal graph, so a GNN models relationships between points directly — a better structural fit than treating features independently.
- **Why not heavier models (ST-GNN, Transformers):** these need far more data (10k+) than the current 2-2.5k dataset; using them now would cause severe overfitting.

7. Technology Stack

Stage	Tools / Models
Preprocessing	FFmpeg, OpenCV
Feature Extraction	MediaPipe (Face Mesh + Pose), Optical Flow (Farneback/RAFT)
Scoring Model	XGBoost / LightGBM (baseline), Standard GNN via PyTorch Geometric (primary)
Explainability	SHAP, rule-based defect mapping
Prompt Optimization	Sentence Transformers (MiniLM), Llama 3.1 8B / Mistral 7B
Dataset Sources	Pexels, Pixabay, Google Veo 3, Nano Banana, Kling AI
Annotation	Label Studio
Serving	FastAPI, Streamlit, AWS S3 / GCS

8. Dataset Plan

- **Size:** 2,000-2,500 videos for the project phase (scalable to 10,000+ if adopted as a full product).
- **Video length:** 5-30 seconds per clip, single continuous shot, no scene cuts.
- **Diversity:** real and AI-generated videos across Indian and foreign demographics, indoor/outdoor settings, varied topics.
- **Labeling:** human raters score each video 1-5 on realism dimensions via Label Studio; Cohen's Kappa checks rater agreement.

9. Computing Power Requirements

Component	Requirement	Notes
GNN training	1x RTX 3060/3070 (8-12GB VRAM)	Free options: Google Colab / Kaggle GPU
XGBoost training	CPU only	No GPU needed, trains in minutes
Feature extraction (MediaPipe)	CPU only	GPU optional, not required
RAM	16GB minimum, 32GB comfortable	For frame batches + landmark data
Storage	50-100GB	Raw videos + frames + feature files (2.5k videos)
Inference (deployment)	CPU or low-tier GPU	Trained models are lightweight to run

Component	Requirement	Notes
LLM prompt rewriting	12-16GB VRAM if self-hosted	Or use API-based inference to skip local GPU

10. Cost Estimation

10.1 Project build cost (one-time)

Item	Estimated Cost
Compute (Colab/Kaggle free tier, or RunPod fallback)	Rs 0 - Rs 300
Dataset generation (Veo/Kling/Nano Banana credits)	Rs 1,500 - Rs 4,000
Real video sourcing (Pexels/Pixabay)	Rs 0
Annotation (Label Studio, self-hosted)	Rs 0
LLM (open-source, self-hosted)	Rs 0 (Rs 500-2,000 if paid API used as fallback)
Storage / hosting (free tiers)	Rs 0
Total estimated project cost	Rs 500 - Rs 3,000

10.2 Cost per single ad video — before vs after this system

	Before (manual trial and error)	After (with this system)
Generation attempts needed	4-8	2-3
Cost per final usable ad	Rs 200 - Rs 1,200	Rs 100 - Rs 450
Review process	Manual, subjective judgment	Automated score + defect explanation

11. Expected Accuracy

A realistic accuracy target for this project phase is 75-85 percent, measured against human realism ratings. Scaling the dataset to 10,000+ videos and adding ensemble scoring can push this to roughly 85-92 percent over time. Claims of 99-100 percent realism detection are not credible for this category of problem and should not be promised to the client.

12. Project Timeline (2-member team)

Weeks	Activity
Week 1-2	Setup, tool learning, dataset collection (2-2.5k videos)
Week 3-4	Labeling via Label Studio, preprocessing pipeline
Week 5	Feature engineering — statistical vectors and graph structures
Week 6-7	Model training — XGBoost baseline, then GNN tuning
Week 8	Explainability (SHAP) and prompt optimization module
Week 9-10	FastAPI + Streamlit integration, end-to-end testing
Week 11-12	Buffer for debugging, final report and presentation

Total estimated duration: 10-12 weeks (about 2.5-3 months), working part-time alongside coursework.

13. Conclusion

This system provides an automated, explainable way to evaluate and improve AI-generated UGC video realism, replacing manual guesswork with a measurable score, clear defect diagnosis, and an auto-improved prompt. The XGBoost plus Standard GNN combination is realistic to build and complete within the current dataset size and team timeline, while remaining a strong, technically sound foundation the company can scale into a full production product with a larger dataset over time.